

Week 2 Assessment Report

Assessment Overview

This project implements API Automation Testing for the SDET Retail Application using:

- Java
 - Rest Assured
 - JUnit 5
 - JSON Schema Validation
 - JDBC Database Validation
 - MySQL
 - Maven
 - TestContainers Dependency (added for future containerized database execution)
-

Framework Design

The framework follows a modular design:

API Framework

- SpecFactory
 - Centralized request and response specifications
 - Authentication token generation
 - Endpoint path management
 - Common response expectations
- ApiConfig
 - Stores Base URL configuration
- AuthConfig
 - Stores authentication related environment variables

Database Framework

- DatabaseConfig
 - Reads database configuration

- DbSupport
 - Creates JDBC connections
 - Performs connectivity validation
 - OrderRepository
 - Database operations
 - Find Order
 - Delete Order
 - Database_Assertions
 - Database validation methods
-

Authentication Implemented

Customer Authentication

Endpoint:

POST /api/auth/login

Purpose:

- Obtain customer bearer token
- Used for cart and customer order flow

Validated:

- Successful login
 - Token extraction
-

OAuth Authentication

Endpoint:

POST /api/oauth/token

Purpose:

- Obtain OPS JWT Token
- Obtain Viewer JWT Token

Validated:

- Token generation
- JWT Header decoding
- JWT Payload decoding
- Role verification
- Scope verification

Verified claims:

- role = OPS
- orders:read
- orders:write

API Key Authentication

Endpoint:

GET /api/partner/orders/{id}

Purpose:

- Partner order access

Validated:

- X-API-Key authentication
- Partner order retrieval

Environment Variable Configuration

Sensitive information was moved from source code to environment variables.

Examples:

- OPS_CLIENT_ID
- OPS_CLIENT_SECRET
- VIEWER_CLIENT_ID
- VIEWER_CLIENT_SECRET
- RETAIL_API_KEY

- DB_HOST
- DB_PORT
- DB_USER
- DB_PASSWORD
- DB_NAME

Benefits:

- Improved security
- Easier maintenance
- Environment independence
- Industry-standard configuration management

Schema Validation

Implemented JSON Schema Validation using:

json-schema-validator

Validated:

Product Schema

GET /api/products/{id}

Product List Schema

GET /api/products

Order Schema

Customer Orders

Secure Order Schema

OPS Orders

Benefits:

- Contract validation
 - Response structure verification
 - Early API defect detection
-

JDBC Database Validation

Implemented JDBC validation against MySQL database.

Validated:

- Database connectivity
- Order existence
- Order number consistency
- Order status consistency

Comparison performed between:

API Response

and

Database Records

to ensure data integrity.

Order Lifecycle Testing

Implemented complete order lifecycle flow:

Create Order

POST /api/secure/orders

Expected Status:

CREATED

Validated:

- Response schema
 - Order ID
 - Order Number
 - Database record creation
-

Allocate Order

POST /api/secure/orders/{id}/allocate

Expected Status:

ALLOCATED

Validated:

- API response
 - Database status update
-

Ship Order

POST /api/secure/orders/{id}/ship

Expected Status:

SHIPPED

Validated:

- API response
 - Database status update
-

Fetch Order

GET /api/secure/orders/{id}

Expected Status:

SHIPPED

Validated:

- Final lifecycle state
- Response correctness

Lifecycle Flow:

CREATE → ALLOCATE → SHIP → FETCH

Negative Testing

Implemented exact status code validation.

401 Unauthorized

Scenarios:

- Missing Token
 - Invalid Token
 - Expired Token
-

403 Forbidden

Scenario:

- Viewer token attempting OPS operation

Expected:

OPS role required

404 Not Found

Scenario:

- Unknown Order ID
-

409 Conflict

Scenario:

Shipping before allocation

Expected:

Cannot move order from CREATED to SHIPPED

Validation:

- `currentStatus = CREATED`
 - `expectedStatus = ALLOCATED`
-

Test Cleanup

Implemented cleanup using:

@AfterEach

Purpose:

- Delete test-created database records

- Maintain database consistency
 - Avoid test data accumulation
 - Ensure repeatable executions
-

Additional Coverage

Implemented:

- Health Endpoint Validation
 - Query Parameter Validation
 - Path Parameter Validation
 - XML Endpoint Validation
 - Product Verification
 - Secure Endpoint Validation
 - JWT Claim Verification
-

Outcome

The framework successfully demonstrates:

- Request and Response Specifications
- JSON Schema Validation
- Bearer Token Authentication
- API Key Authentication
- JDBC Database Validation
- Order Lifecycle Automation
- Negative Testing
- Test Data Cleanup
- Environment Variable Configuration
- Reusable Test Framework Design